

Introduction générale

La gestion des ressources humaines occupe une place centrale dans le fonctionnement d'une organisation moderne. Elle ne se limite plus à l'enregistrement administratif des salariés ; elle couvre aussi le suivi des congés, les demandes internes, la planification du travail, le pointage, la formation, la communication RH, les réclamations, la paie et la traçabilité des décisions. Lorsque ces processus sont répartis entre des fichiers, des échanges informels ou des validations manuelles, les risques augmentent : perte d'information, erreurs de calcul, lenteur de traitement, manque de visibilité pour les managers et difficulté à justifier les décisions.

Le projet présenté dans ce rapport répond à cette problématique par une plateforme web de gestion des ressources humaines développée avec Django. L'application est organisée autour des modules accounts, core et hr. Elle s'appuie sur le système d'authentification de Django, sur des profils applicatifs portant les rôles ADMIN, RESPONSABLE_RH, RESPONSABLE_HIERARCHIQUE et EMPLOYE, sur une base de données SQLite en environnement local, sur des templates HTML et sur des services métier chargés de sécuriser les traitements sensibles.

Une attention particulière a été portée au module Planning, car il représente un besoin opérationnel fort dans une plateforme RH. La version enrichie permet de créer et consulter des shifts, de distinguer les plans normaux des plans permanents, de gérer des récurrences, d'utiliser des vues calendrier, d'appliquer des filtres compacts, de visualiser des indicateurs, de créer des plannings groupés et de relier les shifts au module Présence / Pointage. Ainsi, le calcul des retards, des sorties anticipées et des heures manquantes s'appuie sur la référence de planning réellement associée à l'employé.

Le projet intègre également un assistant intelligent. L'assistant global utilise une logique de récupération de contexte autorisé avant l'appel éventuel à Gemini. L'assistant Planning, quant à lui, peut répondre à des questions opérationnelles sur les conflits, les heures planifiées ou les employés sans planning, et prépare une couche d'action contrôlée pour les utilisateurs RH ou administrateurs. Cette conception évite que le modèle d'IA modifie directement la base de données ; les validations restent assurées par le backend.

La méthodologie adoptée est incrémentale. Le projet commence par l'étude du contexte RH et de l'existant, puis formalise les besoins, conçoit les entités et les scénarios, réalise les modules avec Django et vérifie les traitements par des tests ciblés. Le rapport est structuré en quatre chapitres. Le premier chapitre présente le contexte général, la problématique, les objectifs, l'étude de l'existant et la méthodologie. Le deuxième chapitre analyse les besoins, les acteurs et les cas d'utilisation. Le troisième chapitre expose la conception, l'architecture, le modèle de données et les choix technologiques. Le quatrième chapitre décrit la réalisation, les interfaces, les tests, la validation, le déploiement et les difficultés rencontrées.

Chapitre 1 : Présentation générale du projet

1.1 Introduction

Ce chapitre présente le cadre général du projet. Il situe le besoin RH auquel répond la plateforme, identifie les limites de l'existant, précise les objectifs attendus et explique la démarche de travail adoptée. Il permet ainsi de comprendre pourquoi une application RH centralisée est pertinente et pourquoi le module Planning a nécessité une attention particulière.

1.2 Contexte général

Les services RH manipulent des données nombreuses et sensibles : identité des employés, affectations, responsables hiérarchiques, congés, documents, rémunération, formations, pointage et demandes internes. Dans l'application étudiée, ces informations sont regroupées dans une solution Django unique. Le dossier config contient la configuration, accounts porte l'identité et les rôles, core gère le tableau de bord et l'assistant global, tandis que hr concentre les modèles métier, les formulaires, les vues, les services, les permissions et les tests.

L'application ne se limite pas à un CRUD d'employés. Elle couvre plusieurs processus de bout en bout : soumission et validation d'un congé, déduction du solde, création de documents, notifications, audit, pointage, gestion des points, boutique interne, réclamations, messagerie RH, tâches d'équipe, actualités et paie. Cette couverture fonctionnelle impose une conception rigoureuse pour garantir la cohérence des données et la confidentialité des informations personnelles.

Le module Planning devient le centre opérationnel de l'organisation du temps. Il structure les shifts individuels, les plannings groupés et les plans permanents. Sa liaison avec le pointage permet de passer d'un simple enregistrement de présence à une analyse planifié/réalisé : heures attendues, heures travaillées, retard, sortie anticipée, manque horaire et heures supplémentaires.

1.3 Problématique

La problématique générale peut être formulée ainsi : comment concevoir une plateforme RH web capable de centraliser les processus administratifs, de sécuriser les accès selon les rôles, d'automatiser les traitements critiques et de fournir une vision fiable du planning et de la présence des employés ?

Cette problématique se décline en plusieurs contraintes concrètes. Le système doit empêcher les incohérences de données, par exemple un congé avec une date de fin antérieure à la date de début, un solde négatif, une hiérarchie circulaire ou deux shifts qui se chevauchent pour le même employé. Il doit également empêcher les fuites d'informations : un employé ne doit pas consulter les données privées d'un autre employé, et un manager ne doit voir que son périmètre autorisé.

Enfin, la plateforme doit rester exploitable dans une démonstration réelle. Les écrans ne doivent pas paraître vides, les messages d'erreur doivent être professionnels, les actions doivent être auditées lorsque nécessaire et l'interface doit permettre à un utilisateur RH de comprendre rapidement la situation.

1.4 Étude de l'existant

Dans un fonctionnement manuel, les informations RH sont souvent dispersées dans plusieurs fichiers, courriels ou conversations. Cette organisation crée des difficultés de suivi, car les validations ne sont pas toujours tracées, les documents peuvent être stockés sans contrôle d'accès et les responsables manquent d'indicateurs consolidés.

L'existant applicatif initial avait déjà une base solide : authentification, employés, congés, demandes administratives, documents, notifications, pointage, planning, formations, réclamations, paie et audit. Toutefois, le module Planning devait être renforcé afin de devenir un vrai espace opérationnel. Les nouveaux besoins imposaient des plans permanents, des vues calendrier plus lisibles, une création groupée, des API prêtes pour l'assistant et une intégration plus directe avec le pointage.

La solution actuelle répond à ces limites par une architecture Django structurée et par une séparation des responsabilités. Les règles critiques sont placées dans les modèles, formulaires et services. Les vues orchestrent les workflows et les templates affichent des données déjà filtrées selon le profil connecté.

Limite identifiée	Risque	Réponse de la plateforme
Données RH dispersées	Perte d'information et doublons	Centralisation des employés, demandes, documents, points et pointages dans les modèles Django.
Validation manuelle des congés	Solde incohérent ou chevauchement	Contrôles dans DemandeCongeForm, modèle DemandeConge et services transactionnels.
Planning peu exploitable	Absence de référence pour le pointage	PlanningShift, plans normaux/permanents, vues calendrier, API et lien avec Pointage.
Permissions uniquement visuelles	Accès direct non autorisé	Filtres queryset, décorateurs login_required/role_required et services de périmètre.
Absence d'aide contextuelle	Difficulté à retrouver l'information	Assistant RAG sécurisé, assistant Planning et réponses limitées au contexte autorisé.

Tableau 1.1 : Critique de l'existant et réponse proposée

1.5 Objectifs du projet

Les objectifs du projet sont à la fois fonctionnels, techniques et pédagogiques. Fonctionnellement, il s'agit de proposer une plateforme RH intégrée, capable de couvrir les processus courants d'une organisation. Techniquement, il s'agit d'exploiter les mécanismes de Django pour produire une solution maintenable, testable et extensible. Pédagogiquement, le projet doit être défendable devant un jury : chaque règle importante doit pouvoir être reliée à un modèle, un formulaire, une vue, un service ou un test.

1.6 Méthodologie adoptée

La démarche suivie est incrémentale. Chaque module est analysé, amélioré puis validé. Cette méthode convient à une application déjà existante, car elle évite de réécrire l'ensemble du système et permet de respecter les conventions du dépôt. Pour le Planning, l'analyse a d'abord porté sur les modèles, vues, formulaires, templates et scripts. Les améliorations ont ensuite été consolidées dans les services, l'interface et les tests.

La méthode met aussi l'accent sur la validation. Les commandes Django permettent de vérifier la configuration, les migrations de test et les scénarios critiques. Les tests ciblés du Planning et de l'assistant global ont été exécutés pour confirmer les comportements sensibles : permissions, confidentialité, création de shift, plan permanent, conflits, calcul pointage et fallback Gemini.

Phase	Travaux réalisés
Analyse	Lecture du dépôt, identification des modules, règles métier et contraintes de sécurité.
Spécification	Définition des acteurs, besoins fonctionnels, besoins non fonctionnels et cas d'utilisation.

Phase	Travaux réalisés
Conception	Architecture Django, modèle de données, scénarios dynamiques, API et assistant.
Réalisation	Amélioration du Planning, lien Pointage, vues calendrier, services, assistant et seed demo.
Validation	Exécution de manage.py check, tests ciblés Planning et tests ciblés assistant.
Rédaction	Production du rapport final conforme aux consignes données.

Tableau 1.2 : Planning prévisionnel du projet

1.7 Conclusion

Ce chapitre a présenté le contexte général, la problématique et la démarche adoptée. La plateforme répond à un besoin de centralisation, de fiabilité et de sécurité des processus RH. Le module Planning constitue une évolution importante, car il transforme la gestion du temps en une référence exploitable pour le pointage et pour le pilotage RH. Le chapitre suivant formalise les besoins fonctionnels, non fonctionnels et les principaux cas d'utilisation.

Chapitre 2 : Analyse et spécification des besoins

2.1 Introduction

Ce chapitre présente les besoins de la plateforme. Il identifie les acteurs, décrit les fonctionnalités attendues, précise les exigences non fonctionnelles et détaille les cas d'utilisation les plus importants. L'analyse s'appuie sur le comportement réel du code Django, notamment les modèles, les formulaires, les services, les vues, les routes et les tests.

2.2 Acteurs du système

Le système distingue quatre profils principaux. L'administrateur possède le périmètre le plus large. Le responsable RH gère les opérations RH courantes. Le responsable hiérarchique consulte son équipe et suit les informations rattachées à ses collaborateurs. L'employé consulte ses données personnelles et réalise ses propres actions, comme le pointage ou la soumission de demandes.

Acteur	Rôle dans la plateforme	Accès principal
Administrateur	Supervise les modules sensibles et l'administration générale.	Utilisateurs, paie, audit, référentiels, planning, statistiques.
Responsable RH	Traite les processus RH et pilote les opérations.	Employés, congés, demandes, documents, planning, formations, réclamations.
Responsable hiérarchique	Suit son périmètre d'équipe.	Planning visible, pointages d'équipe, tâches, demandes liées à ses collaborateurs.
Employé	Utilise les services RH pour ses propres données.	Pointage personnel, demandes, formations, boutique, planning personnel, messages RH.

Tableau 2.1 : Acteurs du système

2.3 Besoins fonctionnels

Les besoins fonctionnels couvrent l'ensemble des processus RH. L'application doit authentifier les utilisateurs, gérer les profils actifs, organiser les employés par département, service, poste et responsable, traiter les congés, suivre les documents, calculer les points, permettre le pointage, produire des tableaux de bord et sécuriser la consultation des données selon les rôles.

Pour le Planning, les besoins sont plus spécifiques. L'utilisateur RH doit pouvoir créer un shift individuel ou groupé, sélectionner une cible par service, département, entreprise ou liste d'employés, choisir un statut, définir une pause, consulter les conflits, copier une période, déplacer ou redimensionner un shift normal et accéder à des vues adaptées à différents horizons temporels. Le système doit aussi accepter les plans permanents, utiles pour modéliser le rythme standard d'une entreprise sans imposer une date de fin artificielle.

Module	Besoins fonctionnels
Authentification	Connexion, profil actif, rôles applicatifs, redirection vers le tableau de bord.
Employés et organisation	Création, modification, archivage, affectation à un département, service, poste et responsable.
Congés	Soumission, validation, refus, annulation, solde, mouvement de solde, double validation manager/RH.
Documents	Upload, contrôle d'extension, taille maximale, téléchargement sécurisé et archivage logique.

Module	Besoins fonctionnels
Planning	Shifts normaux, plans permanents, récurrence, filtres, vues calendrier, création groupée, API, conflits.
Pointage	Entrée, sortie, total d'heures, retard, sortie anticipée, heures manquantes, points calculés.
Assistant	RAG sécurisé, refus hors permission, fallback local, assistant Planning pour résumés et conflits.

Tableau 2.2 : Besoins fonctionnels principaux

2.4 Besoins non fonctionnels

- Sécurité : les données doivent être filtrées côté backend selon le rôle et le périmètre de l'utilisateur.
- Fiabilité : les opérations critiques doivent être validées par les modèles, les formulaires ou les services.
- Traçabilité : les actions importantes doivent alimenter l'historique d'audit et les notifications lorsque cela est pertinent.
- Ergonomie : les interfaces doivent être compactes, lisibles, responsives et cohérentes avec le design existant.
- Maintenabilité : les traitements complexes doivent rester dans les services plutôt que dans les templates.
- Testabilité : les comportements sensibles doivent être vérifiés par des tests Django ciblés.
- Confidentialité IA : l'assistant ne doit recevoir que le contexte autorisé et doit refuser les demandes d'accès privé ou les tentatives d'injection.

2.5 Diagramme de cas d'utilisation

Le diagramme suivant synthétise les interactions entre les acteurs et les principaux modules. Il met en évidence la place du Planning comme passerelle entre la gestion du temps, le pointage et les statistiques RH.

Employé	S'authentifier Consulter ses données Pointer entrée/sortie Soumettre demandes Consulter planning
Manager	Consulter équipe Suivre pointages Voir planning d'équipe Suivre tâches
Responsable RH	Gérer employés Traiter demandes Créer planning Gérer documents Analyser indicateurs
Administrateur	Administrer rôles Consulter audit Gérer paie Superviser plateforme
Assistant Gemini	Répondre avec contexte autorisé Aider sur Planning Refuser hors permission

Figure 2.1 : Diagramme global des cas d'utilisation du système RH

2.6 Description des principaux cas d'utilisation

Élément	Description
Acteur principal	Responsable RH ou administrateur.
Objectif	Créer un shift normal ou un plan permanent pour un employé, un service, un département ou un groupe.

Élément	Description
Préconditions	Utilisateur connecté, rôle autorisé, dates valides, employé actif si le shift est assigné.
Scénario nominal	L'acteur choisit la cible, renseigne les horaires, la pause, le statut et enregistre. Le service crée les shifts, audite l'action et notifie l'employé lorsque nécessaire.
Scénarios alternatifs	Dates invalides, pause incohérente, chevauchement, congé validé ou plan permanent déjà actif : la création est refusée avec un message clair.
Postconditions	Le planning est enregistré et apparaît dans les vues calendrier et les API.

Tableau 2.3 : Cas d'utilisation : créer un planning

Élément	Description
Acteur principal	Employé.
Objectif	Enregistrer la présence du jour et calculer les heures réellement travaillées.
Préconditions	Utilisateur connecté, profil lié à un employé, aucun pointage déjà ouvert.
Scénario nominal	L'employé pointe l'entrée. Le système associe le shift planifié si disponible. À la sortie, le service calcule total, retard, sortie anticipée, heures manquantes et points.
Scénarios alternatifs	Double pointage, sortie sans entrée ou sortie avant entrée : l'action est refusée. Si aucun shift n'existe, un fallback sécurisé évite le calcul erroné des heures manquantes.
Postconditions	Le pointage est fermé, le statut est mis à jour et les points sont enregistrés si applicable.

Tableau 2.4 : Cas d'utilisation : pointer l'entrée et la sortie

2.7 Conclusion

Ce chapitre a formalisé les besoins du système et les rôles des utilisateurs. L'analyse montre que la plateforme vise une couverture RH large, mais que le Planning et sa liaison avec le Pointage constituent l'axe le plus structurant des nouvelles fonctionnalités. Le chapitre suivant présente la conception permettant de répondre à ces besoins.

Chapitre 3 : Conception et choix technologiques

3.1 Introduction

Ce chapitre présente la conception technique et fonctionnelle de la plateforme. Il détaille l'architecture Django, le modèle de données, les scénarios dynamiques, la base de données et les choix technologiques. L'objectif est de montrer comment les besoins exprimés sont traduits en composants logiciels cohérents.

3.2 Architecture de l'application

L'architecture suit une organisation en couches. Les templates et fichiers statiques composent l'interface utilisateur. Les routes Django redirigent les requêtes vers les vues. Les vues récupèrent le profil connecté, appliquent les filtres et orchestrent les services. Les formulaires et modèles valident les données. Les services métier regroupent les traitements critiques, notamment les transactions de points, les soldes de congés, le Planning et le Pointage. L'ORM assure la persistance dans SQLite en développement.

Interface utilisateur	Templates Django, Bootstrap, CSS, JavaScript, widgets assistant
Routage	config.urls, hr.urls, core.urls, accounts.urls
Vues	core.views, hr.views, accounts.views
Services métier	hr.services, hr.planning_services, hr.planning_assistant, core.ai_assistant
Données	Modèles Django, ORM, migrations, SQLite local
Services externes	Gemini via google-genai, Brevo en configuration optionnelle

Figure 3.1 : Architecture applicative en couches

3.3 Modèle de données et diagramme de classes

La classe Employe est le pivot du domaine. Elle est reliée aux départements, services, postes, responsables, congés, documents, pointages, shifts, formations, conversations RH, réclamations, rémunérations et comptes de points. Le modèle UtilisateurProfile rattache un compte Django à un rôle et à un employé. PlanningShift représente les shifts et plans permanents. Pointage référence éventuellement un PlanningShift pour permettre les calculs planifié/réalisé.

UtilisateurProfile	role actif employe
Employe	matricule nom/prenom departement service responsable
PlanningShift	plan_type recurrence_rule date_debut/date_fin pause statut
Pointage	shift heure_entree heure_sortie total_heures statut
DemandeConge	dates statut workflow manager/RH solde
ComptePoints	solde_points transactions

Figure 3.2 : Diagramme de classes métier principal

Entité	Attributs clés	Rôle
UtilisateurProfile	user, role, actif, employe	Porte le rôle applicatif et rattache un compte Django à un employé.
Employe	matricule, nom, email, departement, service, responsable	Représente le salarié et la structure hiérarchique.
PlanningShift	titre, employe, date_debut, date_fin, plan_type, recurrence_rule, pause_minutes	Planifie les shifts normaux et les plans permanents.
Pointage	employe, shift, date, heure_entree, heure_sortie, total_heures, statut	Enregistre la présence et calcule le réalisé par rapport au planning.
DemandeConge	type, date_debut, date_fin, statut, employe	Gère les absences, validations et solde de congés.
TacheEquipe	titre, mode_affectation, employe, manager, shift, statut	Suit les tâches opérationnelles et peut être liée à un shift.
ConversationRH	sujet, employe, responsable_rh, statut, priorité	Gère les échanges RH et tickets internes.
HistoriqueAction	action, details, utilisateur, entite	Assure l'audit des actions importantes.

Tableau 3.2 : Dictionnaire de données synthétique

3.4 Diagrammes de séquence

Les deux séquences principales concernent la création d'un shift et le pointage. Elles montrent que l'interface ne modifie pas directement les données : la requête passe par une vue, puis par un service, puis par les validations du modèle.

1	RH/Admin saisit le planning dans l'offcanvas ou le formulaire groupé.
2	La vue <code>planning_api_shifts</code> ou <code>planning_create</code> lit le payload et le profil.
3	<code>create_shift</code> ou <code>bulk_create_shifts</code> vérifie les permissions.
4	<code>PlanningShift.full_clean</code> bloque dates invalides, chevauchements, congés et plan permanent doublon.
5	Le shift est sauvegardé, audité et notifié si nécessaire.

Figure 3.3 : Séquence de création et validation d'un shift

1	L'employé pointe son entrée.
2	<code>pointer_entree</code> recherche le shift normal actif, puis un plan permanent applicable.
3	Le pointage est créé avec la référence <code>PlanningShift</code> si elle existe.
4	À la sortie, <code>pointer_sortie</code> calcule la fenêtre prévue, les heures, le retard et la sortie anticipée.
5	Le commentaire détaille les écarts et les points sont appliqués via <code>TransactionPoints</code> .

Figure 3.4 : Séquence du pointage fondé sur le planning

3.5 Modèle de base de données

Le modèle relationnel repose sur les clés étrangères Django. Les relations les plus importantes sont : UtilisateurProfile vers Employe, Employe vers Departement, Service, Poste et responsable, PlanningShift vers Employe, Departement et Service, Pointage vers Employe et PlanningShift, DemandeConge vers Employe, TacheEquipe vers PlanningShift et Employe, et HistoriqueAction vers UtilisateurProfile.

accounts_utilisateurprofile	1 -- 0..1	hr_employe
hr_employe	1 -- n	hr_planningshift
hr_planningshift	1 -- n	hr_pointage
hr_employe	1 -- n	hr_demandeconge
hr_employe	1 -- 1	hr_comptepoints
hr_comptepoints	1 -- n	hr_transactionpoints

Figure 3.5 : Modèle relationnel synthétique

3.6 Choix technologiques

Les technologies ont été choisies pour leur cohérence avec le dépôt existant et leur adéquation à un projet RH. Django offre un ORM fiable, un système d'authentification, des formulaires, des migrations, des vues et des tests. SQLite facilite le développement local. Bootstrap et Bootstrap Icons accélèrent la réalisation d'une interface professionnelle. google-genai permet l'appel optionnel à Gemini sans exposer la clé dans le code.

Technologie	Utilisation	Justification
Python 3.12.10	Langage principal	Lisible, adapté à Django et aux scripts de gestion.
Django 5.0.6	Framework web	ORM, sécurité, migrations, templates, tests et authentification intégrés.
SQLite	Base locale	Simple pour développement, démonstration et tests.
Bootstrap 5.3	Interface	Composants visuels cohérents et responsive.
Bootstrap Icons	Icônes	Lisibilité des actions et navigation compacte.
JavaScript natif	Interactions Planning	Drag-and-drop, API fetch, assistant, modal et offcanvas sans dépendance lourde.
google-genai	Assistant Gemini	Intégration IA contrôlée par variables d'environnement.
ReportLab / pypdf	Génération du présent rapport	Production du PDF et conservation de la page de garde originale.

Tableau 3.1 : Choix technologiques et justification

3.7 Conclusion

La conception repose sur une séparation claire entre interface, vues, services, validations et modèles. Cette organisation permet de sécuriser les données, de tester les règles critiques et de faire évoluer le Planning sans fragiliser les autres modules. Le chapitre suivant présente la réalisation concrète, les interfaces, les tests et le déploiement.

Chapitre 4 : Réalisation, tests, validation et déploiement

4.1 Introduction

Ce chapitre décrit la réalisation concrète de la plateforme et plus particulièrement l'évolution du module Planning. Il présente l'environnement de développement, les fonctionnalités livrées, les contrôles d'exception, les tests effectués, la stratégie de déploiement et les difficultés rencontrées.

4.2 Environnement de développement

Le projet est exécuté localement sur Windows dans le répertoire HR_django. La configuration Django utilise la langue française, le fuseau horaire Africa/Casablanca, SQLite en base locale, les fichiers statiques dans static et les fichiers téléversés dans media. Les variables liées à Gemini, Brevo et aux limites de contexte sont lues depuis l'environnement.

Élément	Valeur observée
Système	Windows, projet situé dans OneDrive/Desktop/HR_django.
Python	Python 3.12.10.
Framework	Django 5.0.6.
Dépendances principales	Pillow 10.3.0, google-genai 1.51.0.
Base locale	SQLite via db.sqlite3.
Fuseau horaire	Africa/Casablanca.
Variables IA	GEMINI_API_KEY ou GOOGLE_API_KEY, GEMINI_MODEL, GEMINI_TIMEOUT_MS, RAG_MAX_CONTEXT_ITEMS, RAG_MAX_CONTEXT_TOKENS.

Tableau 4.1 : Environnement de développement et d'exécution

4.3 Fonctionnalités réalisées

La plateforme propose un ensemble complet de modules RH : tableau de bord, gestion des employés, organisation interne, congés, demandes administratives, documents, notifications, audit, pointage, planning, tâches d'équipe, formations, messages RH, actualités, boutique interne, réclamations, points, paie et assistant. Les modules communiquent entre eux : un congé validé influence la disponibilité d'un employé, un shift peut être lié à un pointage, une tâche peut être rattachée à un shift et les actions importantes peuvent produire notifications ou audit.

La refonte visuelle centralisée améliore l'expérience utilisateur. La navigation latérale regroupe les sous-modules du Planning, les cartes statistiques sont compactes, les statuts sont colorés et les écrans utilisent des composants cohérents. Cette cohérence est importante pour une application de gestion, car l'utilisateur doit pouvoir scanner rapidement les informations sans être distrait par une interface instable.

4.4 Module Planning

Le module Planning est accessible par une entrée principale et plusieurs sous-onglets : Overview, Calendar, Daily, Weekly, Bi-weekly, Monthly, Timesheets, Shift Management, Attendance, Time Off, Tasks, Approvals, Reports et Settings. Cette organisation donne une vision progressive : synthèse, calendrier, planification opérationnelle, feuilles de temps, congés, tâches, rapports et paramètres.

Le modèle PlanningShift distingue deux types de plans. Le plan normal possède un début et une fin. Il sert aux shifts ponctuels ou limités dans le temps. Le plan permanent représente un rythme standard,

par exemple un horaire lundi-vendredi, et peut fonctionner sans date de fin grâce à `permanent_end_time`. Les récurrences disponibles sont `daily`, `weekdays`, `weekly`, `biweekly` et `monthly`. Le code empêche un employé d'avoir plusieurs plans permanents actifs et refuse les récurrences sur les plans normaux afin d'éviter les ambiguïtés.

L'interface propose des statistiques compactes : shifts planifiés, plans permanents, demandes en attente, retards, congés validés, heures planifiées et alertes. Les filtres incluent période, recherche, employé, département, service, statut, type de planning et niveau de conflit. Les vues calendrier utilisent des blocs colorés, des indicateurs +X more, un `offcanvas` d'édition, un modal de détail, des boutons de redimensionnement et un `drag-and-drop` pour déplacer les shifts normaux.

Vue	Rôle
Overview	Résumé opérationnel, alertes d'heures, indicateurs rapides.
Calendar / Daily / Weekly	Lecture visuelle des shifts par jour, semaine ou employé.
Bi-weekly / Monthly	Vision plus longue pour rotations et plans récurrents.
Timesheets / Attendance	Comparaison entre planning et pointage.
Leave / Tasks	Disponibilités, congés et tâches liées au planning.
Reports / Settings	Synthèses, exports prévus, paramètres et limites actuelles.

Figure 4.1 : Organisation fonctionnelle du module Planning

4.5 Liaison Planning et Pointage

La liaison entre Planning et Pointage est un apport central. Lorsqu'un employé pointe l'entrée, le service `pointer_entree` cherche d'abord un shift normal publié couvrant la période courante. S'il n'en trouve pas, il recherche un plan permanent applicable au jour. Le pointage conserve alors une référence vers `PlanningShift`, ce qui permet au calcul de sortie de comparer le réel au prévu.

À la sortie, `pointer_sortie` calcule la fenêtre attendue à partir du shift associé ou, à défaut, des paramètres officiels de pointage. Le service calcule les heures travaillées, le retard, la sortie anticipée, les heures manquantes et les points. Le commentaire du pointage conserve une trace lisible de ces écarts. Si aucun shift n'est trouvé, la fonction `pointage_breakdown` retourne un avertissement professionnel et ne calcule pas d'heures manquantes artificielles.

4.6 Assistant Gemini

Deux niveaux d'assistance existent. L'assistant global, exposé par `core.views.chatbot_api`, récupère un contexte autorisé selon le rôle de l'utilisateur, refuse les demandes sensibles ou les tentatives d'injection, puis appelle Gemini si la configuration est disponible. En cas d'absence de clé ou d'erreur, il fournit une réponse déterministe fondée sur le contexte récupéré.

L'assistant Planning, exposé par `planning_api_assistant`, est spécialisé dans les besoins opérationnels du planning. Il peut répondre localement aux demandes simples : afficher les conflits, identifier les employés sans planning et résumer les heures planifiées. Pour les actions plus complexes, Gemini est sollicité pour produire une intention structurée, mais l'exécution reste contrôlée par le backend. Les employés ne peuvent pas utiliser l'assistant pour consulter les informations privées d'autres employés, et les mutations sont réservées aux RH/Admin.

4.7 Gestion des exceptions

Catégorie	Traitement réalisé
Dates invalides	Début dans le passé pour plan normal, fin obligatoire, fin avant début et shift de nuit non supporté sont refusés.
Pause incohérente	Pause plus longue que le shift, pause sans durée ou hors intervalle refusée.
Conflits planning	Chevauchement de shifts normaux et congé validé détectés.
Plan permanent	Heure de fin standard obligatoire et un seul plan permanent actif par employé.
Permissions	Création, modification, déplacement et suppression réservés aux RH/Admin.
Pointage sans shift	Avertissement clair et absence de calcul d'heures manquantes erroné.
Assistant	Message vide refusé, accès privé refusé, fallback si Gemini indisponible.
API	Réponses JSON homogènes : ok, data, errors, message.

Tableau 4.2 : Exceptions et validations gérées

4.8 Tests et validation

Les vérifications ont été effectuées directement dans l'environnement du projet. La commande `python manage.py check` a confirmé l'absence de problème système. Les tests ciblés du module Planning ont validé les scénarios critiques : accès par rôle, sous-onglets, création de shift, plan permanent, récurrences, rejet des cas invalides, rapports, modal de détail, lien pointage, permissions, déplacement, redimensionnement, création groupée, copie, conflits, disponibilité et assistant. Les tests ciblés de l'assistant global ont validé la confidentialité, le refus des mutations, la protection contre l'injection, la navigation et les règles d'accès.

Commande ou scénario	Résultat
<code>python manage.py check</code>	Aucun problème système détecté.
<code>python manage.py test hr.tests.PlanningModuleUpgradeTests --verbosity 2</code>	15 tests exécutés, 15 réussis, durée environ 90,5 s.
<code>python manage.py test core.tests --verbosity 2</code>	12 tests exécutés, 12 réussis, durée environ 65,1 s.
<code>python manage.py test</code>	La suite complète a dépassé la fenêtre d'exécution de cinq minutes dans cette session ; elle doit être relancée hors contrainte de temps avant impression finale.

Tableau 4.3 : Tests et vérifications réalisés

Saisie utilisateur	Formulaires HTML / JSON API
Contrôle vue	Profil, rôle, périmètre, CSRF, méthode HTTP
Service métier	Transactions, audit, notifications, parsing sécurisé
Modèle Django	full_clean, contraintes métier, relations ORM
Tests	Cas nominaux, refus, confidentialité, assistant, pointage

Figure 4.2 : Chaîne de validation technique

4.9 Déploiement

Le projet fonctionne localement avec SQLite. Pour un déploiement réel, plusieurs adaptations sont nécessaires : désactiver DEBUG, externaliser SECRET_KEY, configurer ALLOWED_HOSTS, utiliser une base de données serveur comme PostgreSQL, collecter les fichiers statiques, gérer les médias, fournir les variables GEMINI_API_KEY ou GOOGLE_API_KEY si l'assistant doit appeler Gemini, et sécuriser les clés Brevo si les courriels de vérification sont activés.

Le déploiement doit suivre une procédure reproductible : création de l'environnement virtuel, installation des dépendances, migrations, chargement éventuel des données de démonstration, création d'un compte administrateur, puis lancement par un serveur compatible WSGI/ASGI. Les commandes de seed sont utiles en démonstration ; seed_demo_rh crée des données professionnelles, des plannings permanents et des rotations normales avec des identifiants de démonstration.

4.10 Difficultés rencontrées

- Transformer un Planning simple en module opérationnel sans perturber les autres modules RH.
- Garder les validations critiques côté backend afin d'éviter les contournements par API directe.
- Relier le pointage au planning tout en conservant un fallback sécurisé lorsqu'aucun shift n'existe.
- Intégrer Gemini sans exposer de clé API et sans donner au modèle un accès direct à la base de données.
- Préserver la lisibilité de l'interface malgré le nombre élevé de sous-vues, filtres et indicateurs.
- Tester les comportements sensibles dans un environnement local où la suite complète dépasse la fenêtre d'exécution disponible.

4.11 Conclusion

Ce chapitre a présenté la réalisation concrète de la plateforme et les améliorations du module Planning. Les fonctionnalités développées forment un ensemble cohérent : gestion des shifts, plans permanents, filtres, vues calendrier, API, assistant, données de démonstration et calcul de présence basé sur le planning. Les tests ciblés confirment les scénarios principaux et les limites restantes sont clairement identifiées pour les évolutions futures.

Conclusion générale

Ce projet a permis de concevoir et de réaliser une plateforme web de gestion des ressources humaines avec Django. La solution centralise les données et les processus RH essentiels : employés, organisation, congés, demandes administratives, documents, notifications, audit, pointage, planning, tâches, formations, messagerie RH, actualités, boutique interne, réclamations, points et paie. Elle répond ainsi à un besoin réel de centralisation, de fiabilité, de traçabilité et de sécurité.

L'amélioration la plus importante concerne le module Planning. Celui-ci ne se limite plus à une liste de shifts ; il devient un espace de pilotage avec plans normaux, plans permanents, récurrences, statistiques, filtres, calendrier, création groupée, API et assistant spécialisé. Sa liaison avec le module Pointage permet de calculer les écarts entre le temps prévu et le temps réellement travaillé, ce qui rend la présence beaucoup plus exploitable pour les responsables RH.

Les résultats obtenus montrent une application riche et défendable techniquement. Les règles métier importantes sont placées dans des modèles, formulaires et services. Les permissions ne reposent pas uniquement sur l'affichage des boutons ; elles sont également appliquées côté backend. L'assistant Gemini est encadré par une récupération de contexte autorisé, par des refus de sécurité et par un fallback local. Les données de démonstration rendent les interfaces vivantes tout en gardant une logique idempotente.

Certaines limites restent néanmoins présentes. Les exports PDF, CSV et Excel du module Planning sont encore affichés comme non configurés. Le workflow d'approbation Planning est préparé mais non activé. Les shifts de nuit ne sont pas supportés dans l'état actuel ; ils doivent être découpés ou faire l'objet d'une évolution spécifique. La création et modification du Planning sont réservées aux profils RH/Admin, tandis que les managers disposent surtout d'une consultation de leur périmètre. Enfin, le passage en production nécessite une configuration plus stricte que l'environnement local, notamment pour la base de données, les secrets, les médias et les hôtes autorisés.

Les perspectives d'amélioration consistent à ajouter les exports opérationnels, activer un workflow d'approbation, enrichir la gestion des shifts de nuit, permettre des droits de modification limités aux managers selon leur périmètre, améliorer les rapports de présence, renforcer les tests de bout en bout et préparer un déploiement avec PostgreSQL. En conclusion, la plateforme constitue une base solide pour une gestion RH intégrée, moderne et évolutive.

Annexes

Annexe A : Routes Planning principales

Route	Rôle
/planning	Page principale du module Planning et sous-onglets.
/planning/creer	Création groupée via formulaire classique.
/planning/api/shifts	Liste et création de shifts en JSON.
/planning/api/shifts/<id>	Détail, mise à jour ou annulation d'un shift.
/planning/api/shifts/<id>/move	Déplacement d'un shift normal.
/planning/api/shifts/<id>/resize	Redimensionnement d'un shift normal.
/planning/api/bulk	Création groupée par cible.
/planning/api/copy	Copie d'une période de planning.
/planning/api/conflicts	Détection de conflits.
/planning/api/summary	Synthèse statistique.
/planning/api/available-employees	Employés disponibles sur une période.
/planning/api/assistant	Assistant spécialisé Planning.

Tableau A.1 : Routes Planning principales

Annexe B : Commandes utiles

- `python -m venv .venv`
- `.\venv\Scripts\activate`
- `pip install -r requirements.txt`
- `python manage.py migrate`
- `python manage.py seed_demo_rh`
- `python manage.py check`
- `python manage.py test hr.tests.PlanningModuleUpgradeTests --verbosity 2`
- `python manage.py test core.tests --verbosity 2`
- `python manage.py runserver`